AEML - Assignment

Deadline: December 21, 2025 at 23:59

Please find this attached repository.

Vehicular automation is a notable challenging domain where concepts covered in this course can be applied.

In this assignment we have a simple 2D environment with an agent, destination and obstacles to avoid. The obstacles can be either static or moving. For simplicity, all objects are modeled as circles with radius $r = 1$.

`tasks.py` comes in handy when we want to train something like meta-learning.

`base.py` defines an environment which models core elements required in self-driving cars:

- Perception is modeled as M evenly fanned rays casted from the center of the agent.
- Navigation is solved using a dynamic programming algorithm fully executed on a device.

The environment has the following interface

- `init_params()` prepares `env_params` and `init_state`.
- `reset()` can be used to set `env_state` back to `env_state`.
- `step()` executes one step of the simulation.
- `get_observation()` returns the "point-of-view" of the agent.

Note: students are allowed to openly and *conscientiously* discuss and aid each other in solving *Stage 1* and *Stage 2* of this assignment. Students are also allowed to use any external material including AI assistants. **BUT** the following is discouraged:

- Copying, manipulating and claiming someone else's work as their own
- IF you used your classmate's code - cite the author (no penalty if cited)
  - not providing references to the original work done by your colleague is punishable !
- IF you used some external material - also cite the author
  - This rule is a recommendation
  - Additionally, no need to cite AI assistants

Note #2: Assignment submission:

- A Moodle form will be opened where students should attach a link to the GitHub repository containing the implementation of this assignment.
- Students should create a separate branch called *Submission* that will be reviewed. Students are not allowed to modify that branch after the deadline and before receiving their grade ¡ **IMPORTANT** !
- The reports can be either in english or in russian

NOTE #3: Use of AI in reports:

- The language in your report should sound natural, *appropriate* and in your own style
- Do noy use AI to create "your thought"

- Reflection is a crucial part of the grade and should not be spoiled through cheating using AI assistants

Stage 1: Environment (20 points)

In this stage your task will be to design and implement (using JAX)

- a model of vehicle dynamics (which includes a state space and an action space).
- an observation space for the agent
- a reward function

You need to create a new python file in `src/env/` for the new environment. The environment should inherit from `BaseEnv`. You can name it `robotaxi.py`, but it's up to you.

What is expected:

- Actions: acceleration and steering angle
- Forces: friction etc

Further details will be provided later and upon request.

Note: Students are allowed to tinker with the source code, but the manipulation of `base.py` could lead to bugs.

What might help:

- Youtube lecture about dynamic bicycle model: link
- Gymnax ("Reinforcement Learning Environments in JAX"): link
- Brax ("Brax is a fast and fully differentiable physics engine"): link

Deliverable 1

Students should attach a signed and *readable* report file (`.md`, `.tex` or `.pdf` format) with proper formatting in their to-be-submitted github repository. The following four points are expected:

- Describe and justify the dynamics model.
- Describe and justify the reward function.
- Discuss the difficulties you encountered and how you resolved them (reflection).
- Attach the results (video) and explain how they can be reproduced locally.

Stage 2: RL (20 points)

This stage is about Reinforcement Learning experiments designed to make the robo-taxi reach the goal. Students are not expected to deliver "optimal agents". What is expected:

- Student demonstrates the understanding of how they are applying RL
- Student understands basic RL concepts
- Student understands the RL algorithm they are applying (at least superficially)
- Student applies relevant computational optimizations

Note: In this section you are allowed to use Torch if you find JAX too challanging (but JAX is recommended).

What might help:

- Any material explaining or implementing PPO
- The following PPO implemented in JAX in particular: link

Deliverable 2

Students should attach a signed and *readable* report file (`.md`, `.tex` or `.pdf` format) with proper formatting in their to-be-submitted github repository. The following four points are expected:

- Describe the experimental setup and the RL model.
- Describe how you optimized the computations.
- Discuss the difficulties you encountered and how you resolved them (reflection).
- Attach the results (video), provide a snapshot/checkpoint of the model, and explain how they can be reproduced locally.